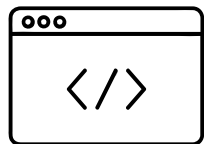
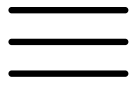


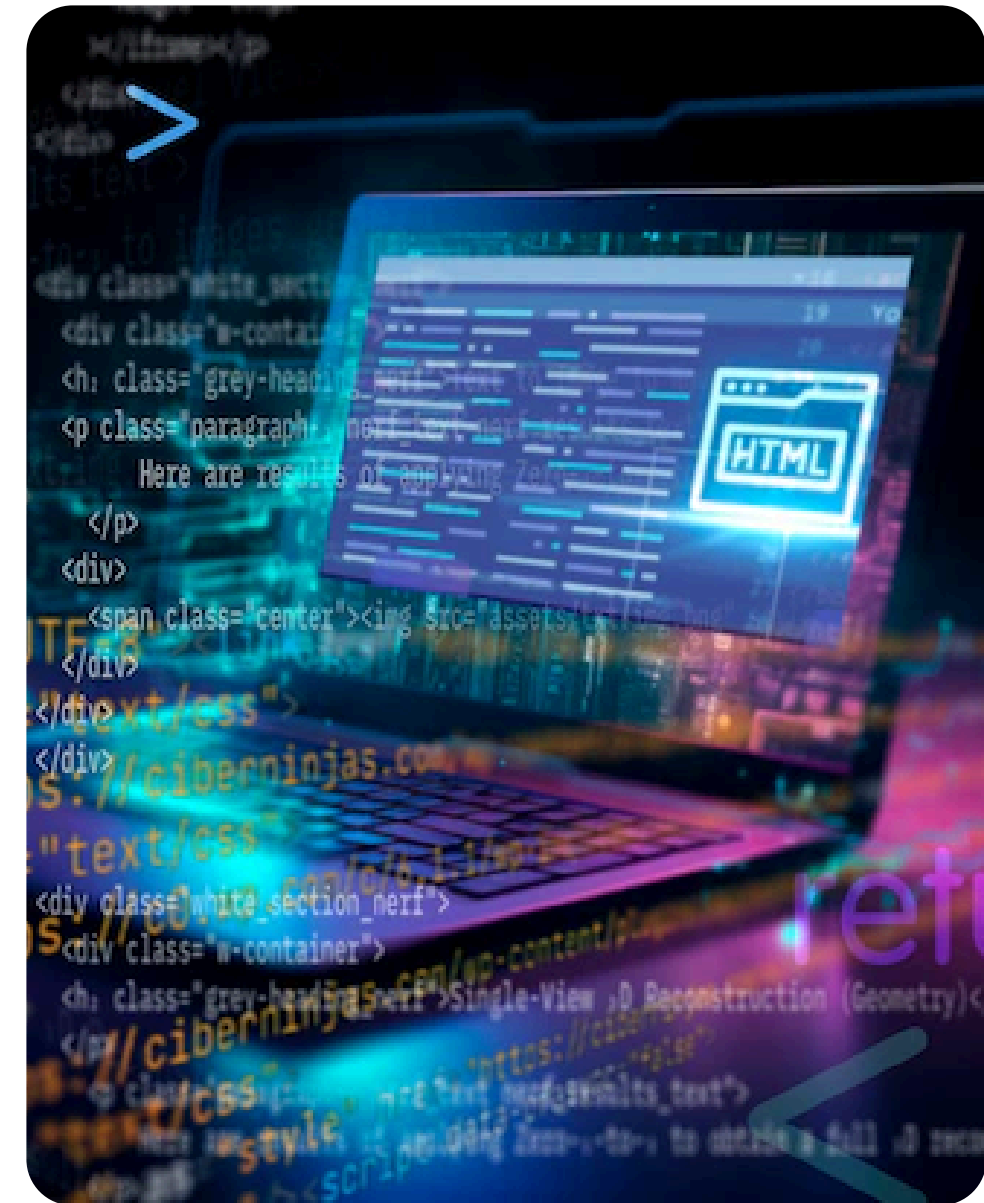
Memoria Dinámica

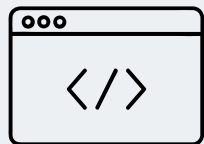
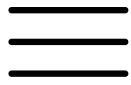
Programación Avanzada



Agenda:

1. Asignar memoria
2. Eliminar memoria
3. Destructor
4. Librería memory





Memoria del sistema

Memoria Estática: Constantes y definiciones.

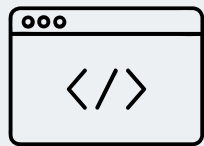
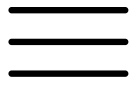
Memoria Automática: Asignada por el sistema para almacenar las variables, constantes, funciones, etc.

Memoria Dinámica: Asignación manual de la memoria a través de apuntadores.

```
#include "ejemplo.h"  
#define valor 10
```

```
int var = 10;  
const c = 100;  
void func(int arg) {}
```

```
int* a = new int(10);  
int* arr = new int[5];  
Contact* p = new Contact(  
    "Jimmy",  
    "3222412343"  
);
```



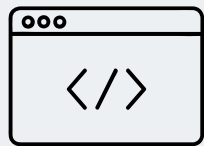
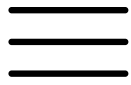
Asignar

Crear un espacio en memoria aleatorio con un tamaño definido.

1. Asignar como variable
2. Asignar como arreglo

```
int* var = new int(10);
```

```
int* arr = new int[5];
```



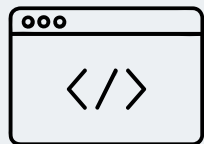
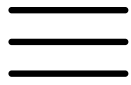
Liberar

Establecer una posición reservada como disponible.

1. Eliminar como variable
2. Eliminar como arreglo

```
int* var = new int(10);  
int* arr = new int[5];
```

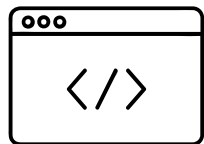
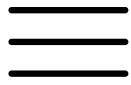
```
delete var;  
delete[] arr;
```



Destructor

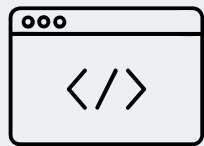
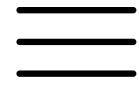
Funcion especial de las estructuras de datos para realizar un comportamiento especifico al eliminar esta de la memoria del sistema.

```
struct Contact {  
    string name;  
    string* numbers;  
    // Constructor  
    Contact() {  
        this->numbers = new string[3];  
    }  
    // Destructor  
    ~Contact() {  
        delete[] this->numbers;  
    }  
};
```



Librería Memory

Programación Avanzada

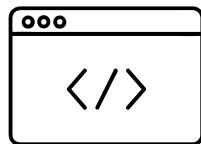
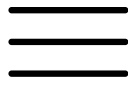


Libreria <memory>

Tipos de datos y funciones destinados a gestionar la memoria de manera eficiente

- `unique_ptr`
- `shared_ptr`
- `weak_ptr`

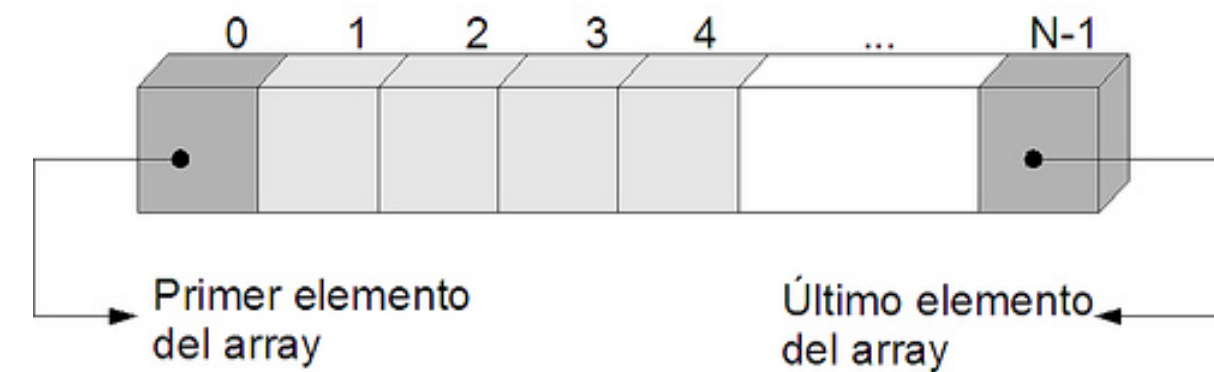
```
int main() {  
  
    unique_ptr<int> uPtr = make_unique<int>(10);  
    unique_ptr<int> uPtr2 = move(uPtr);  
  
    shared_ptr<int> sPtr = make_shared<int>(20);  
    shared_ptr<int> sPtr2 = sPtr;  
    cout<< sPtr.use_count() << endl; // 2  
  
    weak_ptr<int> wPtr = sPtr;  
    cout << *(wPtr.lock()) << endl; // 20  
  
    return 0;  
}
```



Actividad:

Lista: Crear una estructura de datos llamada ListaDinamica la cual se compone de un arreglo el cual se ira escalando a medida que se agreguen o se eliminen elementos de esta.

Hacer gestion correcta de la memoria dinamica del sistema.





Gracias

Programación Avanzada